

DDcentutils API Curriculum

Setup, UI walkthrough, and batch CLI

Contents

1. Get an EMDC API key	1
2. Store the key in <code>.Renviron</code> , not in scripts	1
3. Get the DDcentutils API branch, then verify the key	2
4. R package dependencies	2
Goal	4
Prerequisites	4
Steps	4
Where this connects to the CLI	6
Setup	7
1. One site/scenario, API backend	7
2. Batch: all approved pairs, API, submit-then-resume	7

Do this once on your machine before either the 01 UI walkthrough or the 02 CLI/batch walkthrough.

1. Get an EMDC API key

Sign up at the [EMDC.eco](https://emdc.eco) website and request DayCent researcher access.

Once approved, go to emdc.eco/models/daycent/run. Under **Organization** you'll see which org's credits your runs will draw from; under **API key**, click **Show** to reveal your key (or **Copy** to grab it directly). This is the value used in the `X-Api-Key` header on every API request, and it's tied to your membership in that organization.

Treat it like a password: it grants your account's credit-consuming access.

2. Store the key in `.Renviron`, not in scripts

R reads `~/Renviron` at every session startup and turns each line into an environment variable, without you ever typing the key into a script or the command line. On Windows, confirm which directory R treats as `~`.

```
# Run this in R/RStudio to find the file R actually reads:
Sys.getenv("HOME")
file.path(path.expand("~"), ".Renviron")
```

Open (or create) that exact file and add:

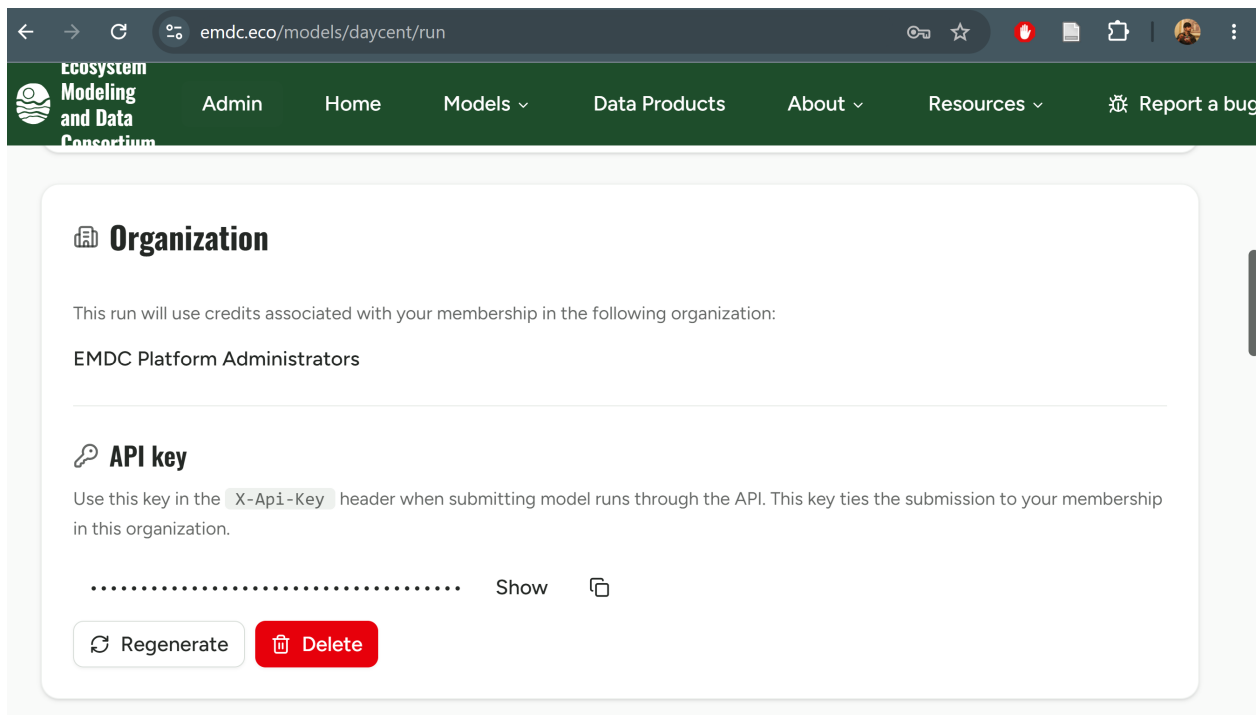


Figure 1: EMDC API key panel

```
EMDC_API_KEY=your-real-key-here
EMDC_BACKEND_SERVER=https://api.emdc.eco
```

Check

- **EMDC_BACKEND_SERVER must include the scheme (https://).** A bare `api.emdc.eco` silently gets 302-redirected by the server.
- Make sure the key itself is current. A stale/revoked key doesn't always fail loudly.

Save the file, then start a **fresh** R session (R only reads `.Renviron` at startup) and verify:

```
Sys.getenv("EMDC_API_KEY") != ""
Sys.getenv("EMDC_BACKEND_SERVER")
```

3. Get the DDcentutils API branch, then verify the key

The API backend lives on a feature branch, not `main`. Clone it and check that branch out:

```
git clone https://github.com/CSU-Soil-Carbon-Solution-Center/DDcentutils.git
cd DDcentutils
git checkout daycent-runner-backends
```

4. R package dependencies

The runner needs `httr`, `stringr`, and `here`, and (for parallel/dev-mode work) `pkgload` and `parallel`. If you're running from a source checkout rather than an installed copy, dev-load the package each session rather than `library(DDcentutils)`:

```
pkgload::load_all(".", quiet = TRUE)
```

You're set up once you have the branch checked out, dependencies loaded, and `.Renvi` configured with a working key.

Goal

Run one site/scenario pair — `wooster` / `cc_ct` — through the UI end to end: upload inputs, preflight, submit, watch status, download results.

Prerequisites

Completed `00-setup-api-key-and-environment.Rmd`: you're logged into the EMDC UI with the same account whose API key you verified.

Steps

1. **Prepare your input files.** The UI needs the same site files the API expects: `wooster_site.100`, `sitepar.in`, `soils.in`, the weather file(s) referenced by the schedule, `wooster_cb_ct.sch`, and `eq_outfiles.in` / `base_outfiles.in` / `scenario_outfiles.in` if you're running equilibrium. You can either zip the `sites/` folder at its top level, or upload the files individually — both are supported.
2. **Enter run details.** Set a descriptive, unique run name (e.g. `wooster-cc_ct-demo`) and an optional description. Model version (`DayCent 491`) is pre-filled — you should not need to supply a product ID directly, the platform resolves that internally from name+version. Choose the run type: **Scenario Run** for a single scenario like this walkthrough's `cc_ct`, **Full Spin-up** for the full equilibrium → baseline → scenario pipeline, or **Batch Run** for multi-site/multi-scenario runs.

Step 1: Run Details

Model version ⓘ
DayCent 491

Run name ⓘ
e.g. Wooster baseline 2020

Description (optional) ⓘ
e.g. calibration run for 2020 baseline

Run Type
Please choose the run type that best fits your needs. If you are new to the platform, we recommend starting with a **Scenario Run** or **Full Spin-up** to verify your file structure. Once you are comfortable, use the **Batch Run** for multi-site simulations or calibration workflows.

Scenario Run
A single scenario analysis.

Full Spin-up ✓
Complete simulation pipeline (equilibrium → baseline → scenario).

Batch Run
Run multi-site and/or multi-scenario simulations.

Figure 2: Run Details screen

3. **Upload your inputs.** Upload either the zipped `sites/` folder or the individual files from step 1 — the platform derives the site name and scenario name from the file naming pattern.

Step 3: File Upload

First, you must enter your site.100 file, then your schedule file(s). Based on the patterns indicated below, we will derive your site name and scenario name.

▼ Required Files

Site Files

☒	[siteName]_site.100	Site-specific parameters	wooster_site.100 (7.69 kB)	×
☒	sitepar.in	Optional Additional site parameters	sitepar.in (2.95 kB)	×
☒	soils.in	Optional Soil properties and parameters	soils.in (977 B)	×

☒ Also include `sitepar.in` and `soils.in`

Schedule Files

☒	Schedule File ([siteName]_[scenarioName].sch)	Scenario-specific parameters	wooster_cc_ct.s... (7.34 kB)	×
-------------------------------------	---	------------------------------	------------------------------	---

Output Files

☒	scenario_outfiles.in	Output format specifications	scenario_outfiles.in (2.05 kB)	×
-------------------------------------	----------------------	------------------------------	--------------------------------	---

Weather Files

☒	Weather (.wth)	Weather data file	Wooster_prism_62.wth (524.7 kB)	×
-------------------------------------	----------------	-------------------	---------------------------------	---

Figure 3: File Upload screen

4. **Review preflight QC and submit.** Before the run starts, the review screen shows the platform's preflight/QC findings (this is the same server-side check the CLI's `--api-dry-run` flag exercises) plus the estimated credit cost. Confirm there are no blocking errors (warnings don't block; errors do) and the cost is acceptable, then click **Submit**.

Step 4: Submit DayCent Run!

Run summary:
Type: Scenario Run Model version: DayCent 491 Run name: ws
Site: wooster Scenario: cc_ct

 Your submission is ready to run!

Estimated Credit Cost 1
A final breakdown of credits used will be available after the run has completed

Warnings

- **sites/wooster/scenario_outfiles.in:** scenario_outfiles.in is missing; the platform default will enable dc_sip.csv.

Info

- **sites/wooster/base_outfiles.in:** base_outfiles.in is missing; platform default output policy will be used for stage 'base'.
- **sites/wooster/eq_outfiles.in:** eq_outfiles.in is missing; platform default output policy will be used for stage 'eq'.
- **sites/wooster/scenario_outfiles.in:** scenario_outfiles.in is missing; platform default output policy will be used for stage 'scenario'.

Changes to be applied

- **sites/wooster/scenario_outfiles.in:** modified

 Submit

Figure 4: Submit review screen with QC findings and credit estimate

5. **Watch status and download results.** Status progresses to **Completed** (or a terminal failure status — if so, stop and read the failure detail rather than resubmitting). Once complete, a **Download all files** button appears on the run detail page. Expect one output set per site/scenario: at minimum a `.bin`, a `_summary.out`, and `_dc_sip.csv` / `harvest` / `year-summary` files depending on your `outfiles.in` selection.

Where this connects to the CLI

Everything above corresponds 1:1 to a `submit_daycent_run()` / `runDayCent_api()` call with `dry_run = TRUE` then `dry_run = NULL` (step 4), `wait = TRUE`, and `download_daycent_results()` (step 5). See `02-batch-cli-r-script.Rmd` for the equivalent scripted version of this exact same run.

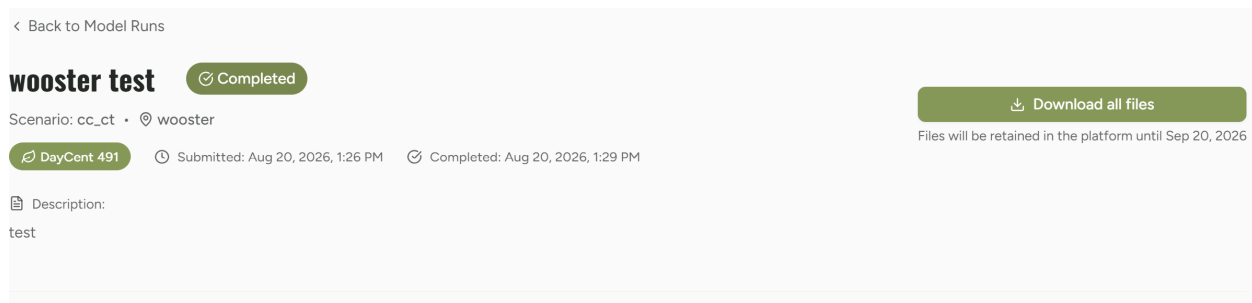


Figure 5: Completed run detail page with Download all files button

Setup

```
pkgload::load_all(".", quiet = TRUE) # dev checkout; use library(DDcentutils) if installed
```

1. One site/scenario, API backend

```
config <- daycent_runner_config(
  "api",
  base_url = "https://api.emdc.eco",
  model_name = "DayCent", model_version = "491"
  # api_key defaults to Sys.getenv("EMDC_API_KEY") - don't pass it literally
)

# 1. Dry run first - no credits spent, no run created.
preflight <- runDayCent_api(
  include = "pendleton/fw_fb_0n_blk6",
  run_eq = FALSE, run_base = FALSE,
  config = config, project_path = here::here(),
  name = "demo-dry-run", wait = FALSE,
  dry_run = TRUE, dry_run_mode = "Full"
)
preflight$passed          # must be TRUE before continuing
preflight$credit_cost     # inspect before spending anything

# 2. Real submission - only after reviewing the estimate above.
result <- runDayCent_api(
  include = "pendleton/fw_fb_0n_blk6",
  run_eq = FALSE, run_base = FALSE,
  config = config, project_path = here::here(),
  name = "demo-run", wait = TRUE # waits for completion + downloads results
)
result$output_paths
```

2. Batch: all approved pairs, API, submit-then-resume

Useful for long-running batches you don't want to block a session on.

```

config <- daycent_runner_config("api", model_name = "DayCent", model_version = "491")
pairs <- c("wooster/cb_nt", "wooster/cb_ct", "wooster/cc_nt", "wooster/cc_ct",
          "pendleton/fw_fb_0n_blk6", "pendleton/fw_nb_45n_blk1")

# Session 1: submit without waiting.
submission <- runDayCent_batch(
  project_path = here::here(), backend = "api", config = config,
  include = pairs, run_eq = FALSE, run_base = FALSE, wait = FALSE
)
parent_id <- submission$parent_run_id
# persist parent_id somewhere (a plain text file is fine - it's not a secret)

# Session 2 (can be a genuinely fresh R session):
config <- daycent_runner_config("api", model_name = "DayCent", model_version = "491")
result <- runDayCent_batch(
  project_path = here::here(), backend = "api", config = config,
  include = pairs, run_eq = FALSE, run_base = FALSE,
  wait = TRUE, parent_run_id = parent_id # resumes and downloads
)
result$task_table

```

Or let the server auto-discover every pair in the uploaded ZIP instead of listing them explicitly:

```

runDayCent_batch(project_path = here::here(), backend = "api", config = config,
  include = "*", run_eq = FALSE, wait = TRUE)

```